

C LANGUAGE

COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ Red star = Important Topic (read carefully, exam / interview favorite)

MMOC TEACH
MADE BY PRAVENDRA

INDEX

- ★ 1. Introduction to C Language
- ★ 2. Variables and Data Types
- 3. Operators
- ★ 4. Input and Output (printf, scanf)
- ★ 5. Conditional Statements (if, else if, else)
- ★ 6. Loops - for, while, do-while
- 7. Break and Continue
- ★ 8. Arrays
- ★ 9. Strings in C (Character Arrays)
- ★ 10. Functions
- 11. Recursion
- ★ 12. Pointers
- 13. Pointers and Arrays
- ★ 14. Structures (struct)
- 15. Unions
- ★ 16. Dynamic Memory Allocation (malloc, calloc, free)
- 17. File Handling
- 18. Storage Classes (static, extern)
- 19. Preprocessor Directives (#define, #include)
- 20. Enum and Typedef

★ 1. Introduction to C Language *[IMPORTANT TOPIC]*

Explanation:

C is one of the oldest and most powerful programming languages. It was created to write system software, like operating systems. Learning C is important because it helps you understand how a computer actually works underneath (memory, pointers, etc.).

A C program is first written as normal text, then 'compiled' (converted into machine code) before it can run.

Key Points:

- Very fast and close to hardware.
- Base for many other languages like C++, Java, and Python.
- Every C program must have a main() function.
- Needs a compiler (like GCC) to run.

Example Code:

```
#include <stdio.h>

int main() {
    printf("Hello, World!");
    return 0;
}
```

Output:

Hello, World!

 *Note: #include <stdio.h> is needed because it gives access to printf() and scanf().*

★ 2. Variables and Data Types *[IMPORTANT TOPIC]*

Explanation:

A variable is a name given to a memory location that stores a value. In C, we must tell the compiler the data type of a variable before we use it.

Key Points:

- int -> whole numbers, example: 10
- float -> decimal numbers, example: 3.14
- char -> a single character, example: 'A'
- double -> bigger decimal numbers

Example Code:

```
int age = 21;
float height = 5.9;
char grade = 'A';

printf("Age: %d, Height: %f, Grade: %c", age, height, grade);
```

Output:

Age: 21, Height: 5.900000, Grade: A

💡 Note: %d is for int, %f is for float/double, %c is for char — these are called 'format specifiers'.

3. Operators

Explanation:

Operators are symbols that perform an action on values, like adding two numbers or comparing them.

Key Points:

- Arithmetic: + - * / %
- Comparison: == != > < >= <=
- Logical: && (and), || (or), ! (not)
- Assignment: = += -= *= /=

Example Code:

```
int a = 10, b = 3;
printf("%d\n", a + b);    // 13
printf("%d\n", a % b);    // 1
```

Output:

```
13
1
```

★ 4. Input and Output (printf, scanf) *[IMPORTANT TOPIC]*

Explanation:

printf() is used to display output on the screen. scanf() is used to take input from the user through the keyboard. Both need format specifiers like %d, %f, or %c to know what type of value they are handling.

Example Code:

```
int age;  
printf("Enter your age: ");  
scanf("%d", &age);  
printf("You are %d years old", age);
```

Output:

```
Enter your age: 21  
You are 21 years old
```

 *Note: scanf() needs an & (address-of symbol) before the variable name, so it knows WHERE to store the input in memory.*

★ 5. Conditional Statements (if, else if, else) *[IMPORTANT TOPIC]*

Explanation:

Conditional statements let the program make decisions. Code inside 'if' runs only when the condition is true. 'else if' checks another condition. 'else' runs when none of the above are true.

Example Code:

```
int marks = 75;

if (marks >= 90) {
    printf("Grade A");
} else if (marks >= 60) {
    printf("Grade B");
} else {
    printf("Grade C");
}
```

Output:

Grade B

★ 6. Loops - for, while, do-while [IMPORTANT TOPIC]

Explanation:

A loop repeats a block of code multiple times. 'for' is used when we know how many times to repeat. 'while' repeats as long as a condition is true. 'do-while' always runs at least once, even if the condition is false.

Example Code:

```
for (int i = 1; i <= 3; i++) {  
    printf("For loop: %d\n", i);  
}  
  
int n = 1;  
while (n <= 3) {  
    printf("While loop: %d\n", n);  
    n++;  
}
```

Output:

```
For loop: 1  
For loop: 2  
For loop: 3  
While loop: 1  
While loop: 2  
While loop: 3
```


7. Break and Continue

Explanation:

'break' stops the loop completely. 'continue' skips the current step and moves on to the next round of the loop.

Example Code:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) continue;  
    if (i == 5) break;  
    printf("%d\n", i);  
}
```

Output:

```
1  
2  
4
```

★ 8. Arrays [IMPORTANT TOPIC]

Explanation:

An array stores multiple values of the same data type together, in one variable, with a fixed size. Each value can be accessed using its index, starting from 0.

Example Code:

```
int numbers[4] = {10, 20, 30, 40};

printf("%d\n", numbers[0]);

for (int i = 0; i < 4; i++) {
    printf("%d ", numbers[i]);
}
```

Output:

```
10
10 20 30 40
```

★ 9. Strings in C (Character Arrays) *[IMPORTANT TOPIC]*

Explanation:

C does not have a separate String type like other languages. Instead, a string is stored as an array of characters, ending with a special hidden character called the 'null character' (\0).

Example Code:

```
char name[20] = "Pravendra";  
printf("Hello, %s", name);  
printf("\nLength: %lu", strlen(name));
```

Output:

```
Hello, Pravendra  
Length: 9
```

 *Note: You must #include <string.h> to use functions like strlen(), strcpy(), and strcat().*

★ 10. Functions [IMPORTANT TOPIC]

Explanation:

A function is a reusable block of code that performs one task. Instead of writing the same code again and again, we write it once and call it whenever needed. This makes the program shorter and easier to manage.

Example Code:

```
int addNumbers(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int total = addNumbers(5, 7);  
    printf("Total: %d", total);  
    return 0;  
}
```

Output:

Total: 12

11. Recursion

Explanation:

Recursion is when a function calls ITSELF to solve a smaller version of the same problem. Every recursive function needs a 'base case' — a condition that stops it from calling itself forever.

Example Code:

```
int factorial(int n) {  
    if (n == 0) return 1;    // base case  
    return n * factorial(n - 1);  
}  
  
printf("%d", factorial(4));
```

Output:
24

★ 12. Pointers [IMPORTANT TOPIC]

Explanation:

A pointer is a special variable that stores the MEMORY ADDRESS of another variable, instead of storing a value directly. Pointers are one of the most powerful (and tricky) features of C.

Example Code:

```
int age = 21;
int *ptr = &age;    // ptr stores the address of age

printf("Value: %d\n", *ptr);    // *ptr gives the value at that address
printf("Address: %p", ptr);
```

Output:

```
Value: 21
Address: 0x7ffeeb... (some memory address)
```

💡 Note: & means 'address of', and * (when used with a pointer) means 'value at that address'.

13. Pointers and Arrays

Explanation:

In C, the name of an array actually acts like a pointer to its first element. This is why arrays and pointers are closely connected, and we can move through an array using a pointer.

Example Code:

```
int numbers[3] = {10, 20, 30};  
int *ptr = numbers;    // points to the first element  
  
printf("%d\n", *ptr);    // 10  
printf("%d", *(ptr + 1)); // 20 (next element)
```

Output:

```
10  
20
```

★ 14. Structures (struct) [IMPORTANT TOPIC]

Explanation:

A structure lets us group different types of related data together under one name. For example, a Student structure can hold a name (text), age (number), and marks (number) all together.

Example Code:

```
struct Student {  
    char name[20];  
    int age;  
};  
  
struct Student s1 = {"Pravendra", 21};  
printf("%s is %d years old", s1.name, s1.age);
```

Output:

```
Pravendra is 21 years old
```


15. Unions

Explanation:

A union looks similar to a structure, but ALL its members share the same memory location. This means a union can hold only one value at a time, which saves memory compared to a structure.

Example Code:

```
union Data {  
    int i;  
    float f;  
};  
  
union Data d;  
d.i = 10;  
printf("%d", d.i);
```

Output:

10

★ 16. Dynamic Memory Allocation (malloc, calloc, free) [IMPORTANT TOPIC]

Explanation:

Normally, arrays have a fixed size decided while writing the code. Dynamic memory allocation lets us create memory space WHILE the program is running, using functions like malloc() and calloc(). We must free this memory manually using free() once we are done with it.

Example Code:

```
int *numbers = (int*) malloc(3 * sizeof(int));
numbers[0] = 10;
numbers[1] = 20;

printf("%d %d", numbers[0], numbers[1]);

free(numbers);    // release the memory back
```

Output:

```
10 20
```

 *Note: Forgetting to use free() causes a 'memory leak' — memory that stays used even though it is no longer needed.*

17. File Handling

Explanation:

File handling in C is used to save data into a file, or to read data from a file, using functions like `fopen()`, `fprintf()`, `fscanf()`, and `fclose()`.

Example Code:

```
FILE *fp = fopen("notes.txt", "w");
fprintf(fp, "Hello File!");
fclose(fp);

FILE *fp2 = fopen("notes.txt", "r");
char text[50];
fscanf(fp2, "%s", text);
printf("%s", text);
fclose(fp2);
```

Output:

Hello

 *Note: Always close a file with `fclose()` after you are done using it.*

18. Storage Classes (static, extern)

Explanation:

A storage class controls the life and visibility of a variable. 'static' keeps a variable's value between multiple function calls instead of resetting it each time. 'extern' lets a variable be shared and used across multiple files.

Example Code:

```
void counter() {  
    static int count = 0;    // keeps its value between calls  
    count++;  
    printf("%d\n", count);  
}  
  
counter();  
counter();  
counter();
```

Output:

```
1  
2  
3
```

19. Preprocessor Directives (#define, #include)

Explanation:

Preprocessor directives are instructions that run BEFORE the actual code is compiled. #include adds another file's code (like stdio.h). #define creates a constant value that gets replaced everywhere in the code automatically.

Example Code:

```
#include <stdio.h>
#define PI 3.14159

int main() {
    printf("Value of PI: %f", PI);
    return 0;
}
```

Output:

Value of PI: 3.141590

20. Enum and Typedef

Explanation:

'enum' lets us create a set of named constant values, which makes code more readable (for example, naming days of the week). 'typedef' lets us give a new, simpler name to an existing data type.

Example Code:

```
enum Day {MON, TUE, WED, THU, FRI};
enum Day today = WED;
printf("%d\n", today);    // WED is the 3rd value, so prints 2 (starts from 0)

typedef unsigned int uint;
uint age = 21;
printf("%u", age);
```

Output:

```
2
21
```